

# 1. Applikations-Architektur

## 1.1 Schichtenarchitektur

Die Anwendung ist in drei Schichten aufgeteilt:

- Presentation Layer (Controller): TourController, TourLogController, AuthController — empfangen HTTP-Requests und delegieren an den Business Layer
- Business Layer (Service): TourService, TourLogService, AuthService, OpenRouteService — enthält die gesamte Business-Logik inkl. Computed Attributes, Full-Text Search und Import/Export
- Data Access Layer (Repository): TourRepository, TourLogRepository, UserRepository — ausschließlich Datenbankzugriffe via Spring Data JPA/Hibernate

Jeder Layer kommuniziert nur mit dem unmittelbar darunterliegenden Layer. Controller rufen nur Services auf, Services nur Repositories. Eigene Exception-Klassen (TourNotFoundException, LocationNotFoundException, InvalidCredentialsException) werden pro Layer definiert und durch einen GlobalExceptionHandler in HTTP-Responses übersetzt.

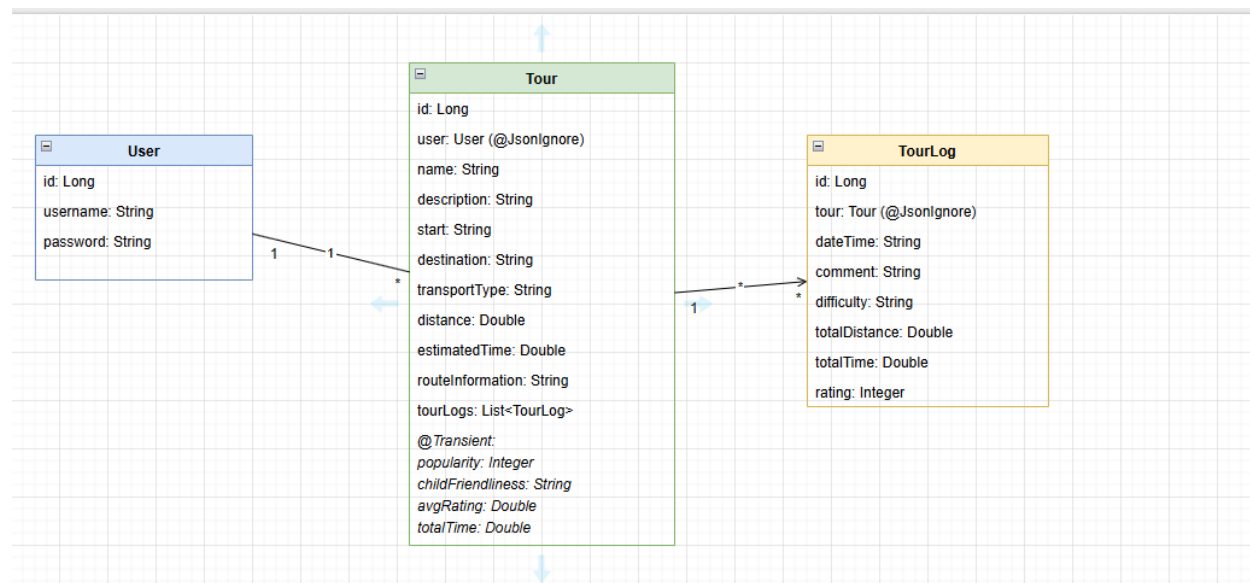
## 1.2 Frontend-Architektur (MVVM)

Das Frontend implementiert das MVVM-Pattern mit Angular 19:

- Model: TypeScript Interfaces (tour.model.ts, tourlog.model.ts, auth.model.ts)
- View: Angular Templates (HTML) mit Angular Material Komponenten
- ViewModel: Angular Components + Services mit Signals für reaktiven State

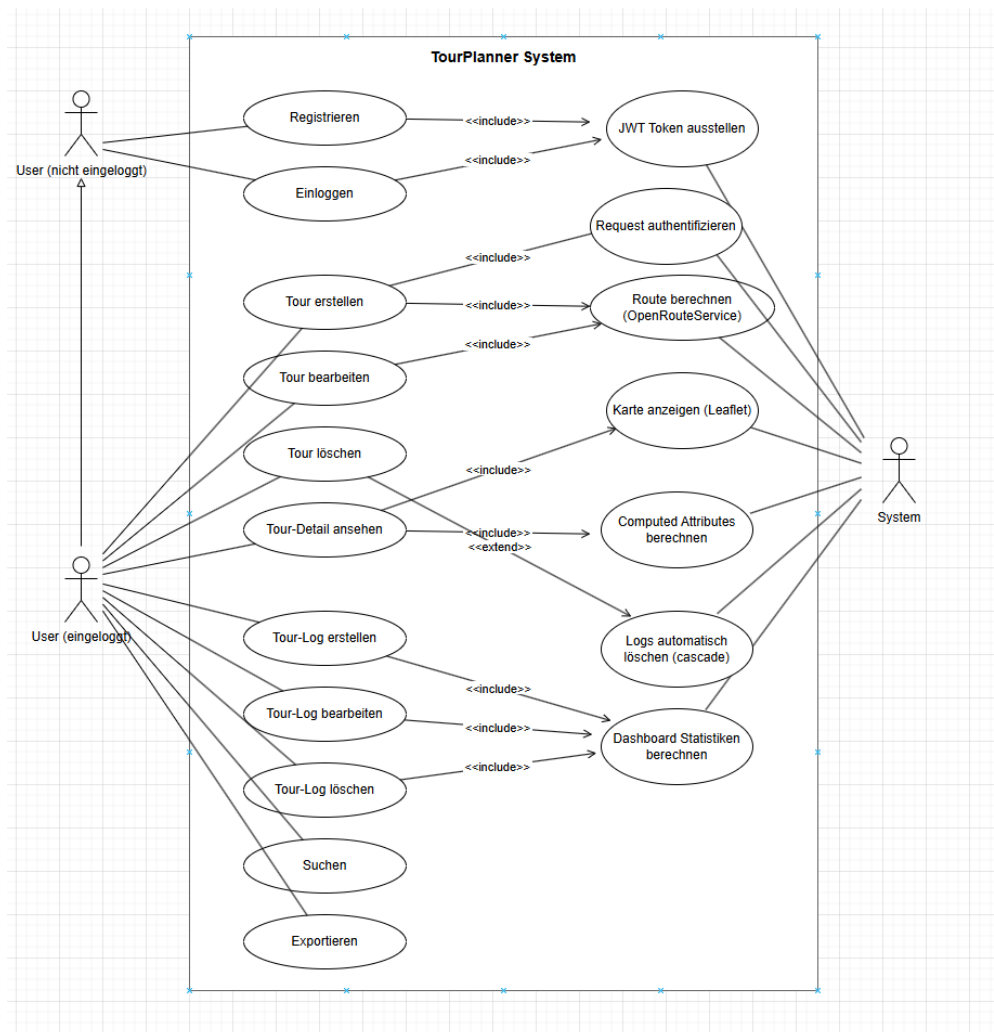
Der globale State wird zentral über TourStateService und TourLogStateService mit Angular Signals verwaltet. HTTP-Calls laufen über TourService und TourLogService, die nach erfolgreichen Requests den State aktualisieren.

## 1.3 Klassendiagramm



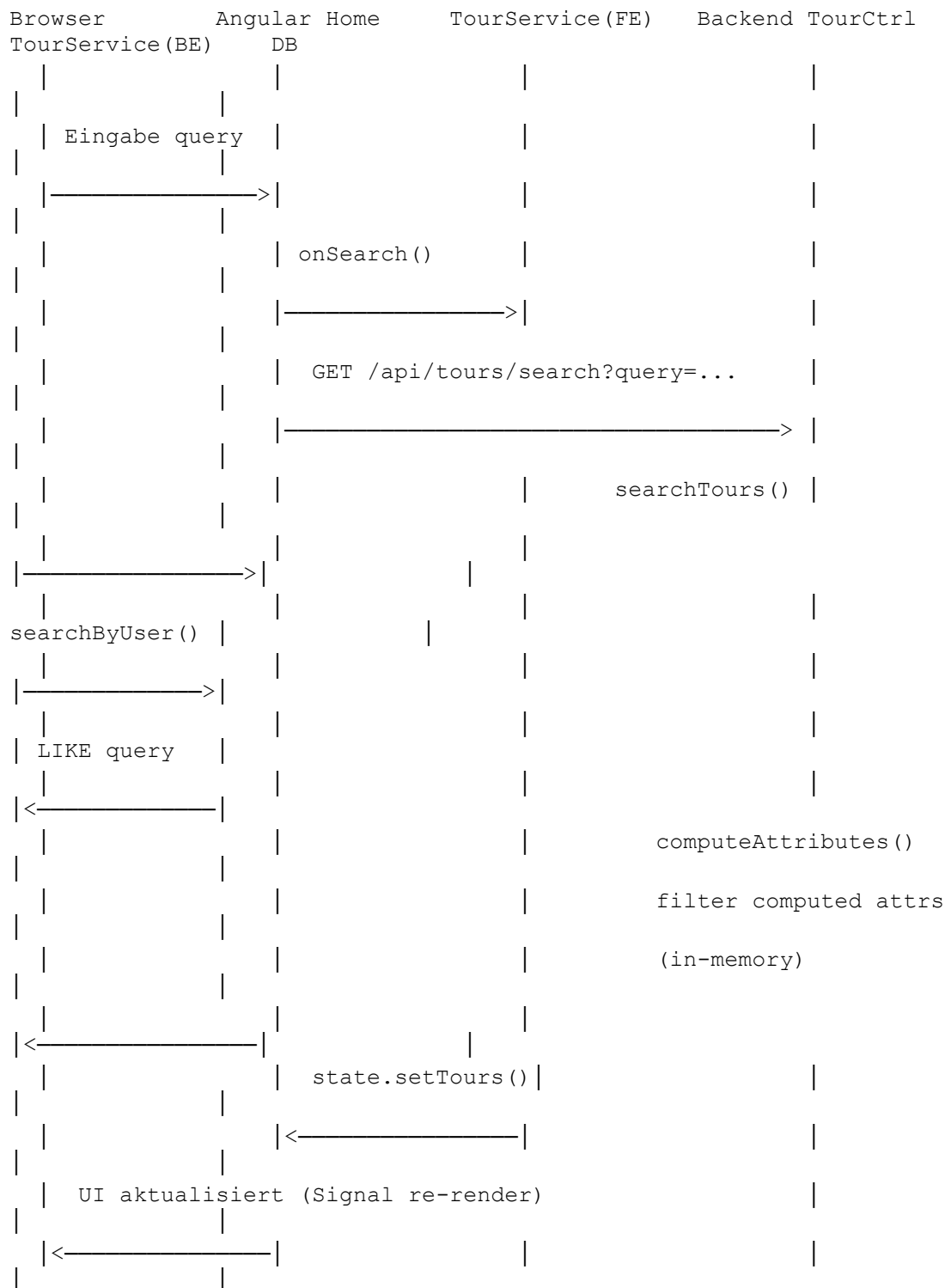
## 2. Use Cases

### 2.1 Use-Case Diagramm



### 2.2 Sequenzdiagramm – Full-Text Search

Ablauf einer Suchanfrage vom Frontend bis zur Datenbankabfrage und zurück:



Die Datenbankabfrage sucht in Tour-Feldern (Name, Beschreibung, Start, Destination, Transportmittel) und TourLog-Feldern (Kommentar, Difficulty) via JPQL LIKE-Query. Der Filter auf Computed Attributes (childFriendliness, popularity) erfolgt danach in-memory im TourService, da @Transient-Felder nicht in JPQL-Queries verwendet werden können.

## 3. Library-Entscheidungen & Lessons Learned

### 3.1 Library-Entscheidungen

| Entscheidung        | Gewählt                                           | Begründung                                                                                 |
|---------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------|
| State Management    | Angular Signals                                   | Moderner als RxJS BehaviorSubject, weniger Boilerplate, direkt in Angular 19 integriert    |
| Auth                | JWT + Spring Security                             | Zustandslos, gut für REST APIs — weit verbreitet und gut dokumentiert                      |
| DateTime            | String statt LocalDateTime                        | Spring Boot 4 / Jackson 3.x hatte Kompatibilitätsprobleme mit LocalDateTime-Serialisierung |
| Computed Attributes | @Transient                                        | Werden immer frisch aus Logs berechnet, keine Sync-Probleme mit DB                         |
| Geocoding           | URLEncoder.encode statt UriComponentsBuilder      | UriComponentsBuilder double-encoded Sonderzeichen wie Leerzeichen in Ortsnamen             |
| Exception Handling  | GlobalExceptionHandler + eigene Exception-Klassen | Saubere Layer-Trennung, HTTP-Status-Codes korrekt                                          |
| Logging             | SLF4J/Logback mit logback-spring.xml              | Flexibel konfigurierbar, in Spring Boot integriert                                         |
| N+1 Problem         | EntityGraph in TourRepository                     | Verhindert N+1 SELECT Problem beim Laden von Touren mit Logs                               |

### 3.2 Lessons Learned

- Jackson 3.x vs 2.x Konflikt: Spring Boot 4 verwendet Jackson 3.x (tools.jackson), aber jjwt-jackson bringt Jackson 2.x mit. Lösung: DateTime als String speichern statt als LocalDateTime.
- Leaflet Map nach Reload leer: ngAfterViewInit wird vor Backend-Daten aufgerufen. Lösung: viewInitialized-Flag das sicherstellt dass die Karte erst nach View-Initialisierung UND Tour-Daten gezeichnet wird.
- State-Verschmutzung beim Tour-Wechsel: Alte Logs blieben im State. Lösung: clearLogs() vor jedem loadLogs()-Aufruf.
- TourLog id=0 Problem: Frontend schickte id:0 mit, Hibernate interpretierte das als existierendes Objekt. Lösung: id bei Creates weglassen, nur bei Updates mitsenden.
- N+1 Problem: Beim Laden aller Touren wurde für jede Tour ein separater SQL-Query für die Logs ausgeführt. Lösung: EntityGraph Annotation im TourRepository für eager loading in einem Query.

## 4. Design Pattern

### 4.1 Repository Pattern (primäres Pattern)

Das Repository Pattern abstrahiert den Datenbankzugriff hinter einer definierten Schnittstelle:

- `TourRepository` extends `JpaRepository<Tour, Long>`
- `TourLogRepository` extends `JpaRepository<TourLog, Long>`
- `UserRepository` extends `JpaRepository<User, Long>`

Der Service Layer kennt keine SQL-Details. Repositories können für Unit Tests einfach gemockt werden (Mockito). Spring Data JPA generiert Implementierungen automatisch aus Methodennamen (z.B. `findByUser`, `findByIdAndUser`, `searchByUser`).

### 4.2 Observer Pattern (Frontend)

Angular Signals implementieren das Observer Pattern. `TourStateService` und `TourLogStateService` halten den globalen State als Signals. Alle Komponenten die `state.tours()` lesen werden automatisch neu gerendert wenn sich der State ändert — ohne manuelles Event-Handling oder Subscriptions.

## 5. Unit Testing Entscheidungen

| Testklasse         | Tests | Warum kritisch                                                                                                    |
|--------------------|-------|-------------------------------------------------------------------------------------------------------------------|
| TourServiceTest    | 17    | Komplexeste Business-Logik: computeAttributes mit Score-Berechnung, User-Isolation (Sicherheit), Full-Text Search |
| AuthServiceTest    | 6     | Sicherheitskritisch: falsches Passwort muss abgelehnt werden, doppelter Username muss verhindert werden           |
| JwtUtilTest        | 5     | Basis der gesamten Authentifizierung: ungültige und abgelaufene Tokens müssen zuverlässig erkannt werden          |
| TourLogServiceTest | 7     | Sicherheitschecks: Log darf nur gelöscht/bearbeitet werden wenn es zur korrekten tourId gehört                    |
| Gesamt             | 35    |                                                                                                                   |

Bewusst nicht getestet wurden: einfache Repository-Methoden ohne eigene Logik (Spring Data JPA ist selbst getestet), Angular-Komponenten (Fokus auf Backend Business-Logik).

## 6. Unique Feature – Dashboard & Meilensteine

Das Dashboard auf der rechten Seite der Startseite zeigt aggregierte Statistiken über alle Touren und Tour-Logs des eingeloggten Users in Echtzeit:

| Statistik              | Berechnung                                                         |
|------------------------|--------------------------------------------------------------------|
| Geplante Touren        | Anzahl aller Touren des Users                                      |
| Geschriebene Logs      | Summe aller popularity-Werte (= Gesamtanzahl Logs)                 |
| Zurückgelegte Distanz  | Summe aller tour.distance Werte in km                              |
| Durchschnitts-Rating   | Durchschnitt aller avgRating-Werte (nur Touren mit Logs)           |
| Beliebteste Tour       | Tour mit dem höchsten popularity-Wert                              |
| Gesamte Aktivitätszeit | Summe aller totalTime-Werte, formatiert als h/min via DurationPipe |

Alle Werte werden als Angular Signals (computed()) im Home-Component berechnet und aktualisieren sich automatisch wenn sich der TourState ändert.



## 7. Zeiterfassung

| Aufgabe                                                           | Luca | Christopher |
|-------------------------------------------------------------------|------|-------------|
| Anforderungsanalyse & Planung                                     | 3h   | 3h          |
| Projektsetup (Angular, Spring Boot, Docker, CORS)                 | 4h   | 5h          |
| Datenbankdesign & Entity-Modellierung                             | 3h   | 3h          |
| Backend: Entities, Repositories, Services, Controller (Tour CRUD) | 5h   | 6h          |
| Backend: TourLog CRUD                                             | 4h   | 5h          |
| JWT Authentication Backend                                        | 4h   | 4h          |
| JWT Authentication Frontend (Interceptor, Guard, State)           | 4h   | 3h          |
| OpenRouteService Integration (Geocoding + Routing)                | 3h   | 6h          |
| Leaflet Map Integration                                           | 2h   | 5h          |
| State Management mit Angular Signals (Tour + TourLog)             | 6h   | 3h          |
| Tour-Log CRUD Frontend + DateTime-Fix                             | 5h   | 4h          |
| Computed Attributes (Popularity, Child-Friendliness, Rating)      | 5h   | 3h          |
| Full-Text Search (Backend + Frontend)                             | 5h   | 3h          |
| Import/Export (JSON)                                              | 2h   | 5h          |
| Dashboard / Unique Feature                                        | 3h   | 6h          |
| Home Page Redesign & Layout                                       | 2h   | 5h          |
| Exception Handling & GlobalExceptionHandler                       | 2h   | 4h          |
| Logging (Logback, SLF4J)                                          | 1h   | 3h          |
| N+1 Problem Fix (EntityGraph)                                     | 1h   | 3h          |
| User-Isolation (Tours pro User)                                   | 3h   | 2h          |
| Input-Validierung Frontend                                        | 2h   | 3h          |
| Unit Tests (35 Tests)                                             | 7h   | 3h          |
| Bugfixing (Leaflet, State, Jackson, Geocoding)                    | 5h   | 5h          |
| UI/UX, Styling, Responsive Design                                 | 2h   | 4h          |
| Cross-Tab Auth Sync & Auto-Logout                                 | 1h   | 3h          |
| Protokoll & Dokumentation                                         | 5h   | 3h          |
| Gesamt                                                            | ~89h | ~101h       |

**Gesamtaufwand Team: ~190h**

## 8. Git-Repository

<https://github.com/itzlucx/semester-project-swen2>